AI QA PLATFORM

Enterprise Quality Assurance & Test Management SaaS — Master Blueprint

EXECUTIVE OVERVIEW

This master document provides a comprehensive end-to-end breakdown of the platform purpose, business requirements, system architecture, core modules, and the step-by-step user journey from initial setup to production release sign-off.

1. The Core Idea & Purpose

CONTEXT

Modern software teams struggle with fragmented quality assurance tools: test cases live in spreadsheets, execution logs are buried in TestRail, bugs are scattered across Jira, and automated results disappear inside CI/CD pipelines. This results in slow release cycles, undetected regressions, and high engineering friction.

The Artificial Intelligence QA Platform solves this by providing a unified, single-pane-of-glass test management solution designed for modern web, mobile, and API testing. It pairs traditional structured QA governance with cutting-edge AI

assistance to streamline the entire software testing lifecycle.

Key Value Propositions

- **Unified QA Workspace:** Eliminates tool sprawl by bringing test repository, plans, runs, defect tracking, and analytics into one connected ecosystem.
- **AI-Accelerated Productivity:** Automates repetitive manual testing work: generates test cases from user stories, diagnoses failure root causes, and detects duplicate bug reports.
- **Enterprise Governance:** Multi-tenant organization isolation, strict Role-Based Access Control (RBAC), immutable audit logging, and quality gates.
- **End-to-End Traceability:** Links every bug directly to the exact test case step, test run, browser session, and release milestone where it

occurred.

2. System Architecture & Tech Stack

ARCHITECTURE

The platform is built as a high-performance, modular monorepo leveraging enterprise best practices:

Technology Stack

- **Frontend Web App:** Next.js 16 (App Router), React 19, Tailwind CSS, Lucide Icons, TanStack React Query.
- **Backend API Service:** Express.js, TypeScript, Prisma ORM, Argon2id encryption, Zod payload validation.
- **Database Layer:** PostgreSQL 16 relational database with relational integrity, foreign keys, and cascading indexes.
- **Async Task Engine:** Redis & BullMQ worker queue for background AI generations, report exports, and webhook delivery.

- **Shared Packages:** @ai-qa/shared-types, @ai-qa/validation, @ai-qa/api-client, and @ai-qa/ui.

Multi-Tenancy & Security Isolation

Every resource (Projects, Test Cases, Test Runs, Defects, API Keys) is strictly scoped to an Organization ID and Project ID. The backend enforces authentication middleware (JWT / Session Cookie) and authorization checks on every single route to guarantee strict cross-tenant isolation.

3. Core Platform Modules

MODULES

Module 1: Multi-Tenant Auth & User Management

Supports multi-tier organizations with 6 granular user roles:

- **SUPERADMIN:** Platform-wide administrator with full system, tenant, and billing control.
- **OWNER / ADMIN:** Full control over organization settings, projects, billing subscriptions, and team invites.
- **QA LEAD:** Can create projects, define test plans, approve AI test cases, and sign off on release milestones.
- **QA TESTER:** Authors test cases, executes test runs, logs defect tickets, and verifies bug fixes.
- **DEVELOPER:** Views linked test cases, updates defect statuses, comments on bugs, and investigates reproduction steps.

- **VIEWER:** Read-only stakeholder access to executive dashboards, pass rates, and coverage reports.

Module 2: Project Management

Projects represent distinct software products (e.g. "Micky Mobile", "Web Portal", "Payment Service"). Each project has a unique key prefix (e.g. MI), environments (Dev, QA, Staging, Prod), team assignments, and target quality objectives.

Module 3: Test Case Repository

Structured hierarchical storage for all manual and automated tests. Features include:

- **Hierarchical Test Suites:** Organize test cases into nested folders by functional domain (e.g., Auth, Cart, Checkout, Settings).
- **Step-by-Step Editor:** Each case contains preconditions, priority, execution time, and ordered steps (Action, Test Data, Expected Result).
- **Immutable Version History:** Every modification increments the version (v1 -> v2) to preserve exact historical run integrity.
- **CSV Bulk Import/Export:** Quickly import thousands of existing test cases from legacy spreadsheets or other tools.

4. Execution Engine & Defect Lifecycle

EXECUTION

Module 4: Milestones & Test Plans

Test Plans group test cases together for targeted testing cycles (e.g., "Sprint 24 Regression Plan", "iOS 18 Compatibility Plan"). Milestones define delivery release dates and aggregate pass rates across all planned executions.

Module 5: Test Execution Workbench

The live execution cockpit for QA testers. Features real-time multi-step verification:

- **Step-Level Verification:** Testers mark each individual step as Passed, Failed, Blocked, or Skipped.
- **Actual Observations & Timers:** Record real observed outcome text, failure reasons, and stopwatch time taken.

- **Multi-Attempt History:** Supports retests with version-tracked execution attempts.
- **1-Click Defect Generation:** If a step fails, a pre-filled bug report is generated with steps to reproduce already attached.

Module 6: Defect & Bug Tracking Lifecycle

Comprehensive bug tracking designed specifically for QA-to-Developer handoffs:

- **Standard Lifecycle:** NEW -> IN_PROGRESS -> READY_FOR_RETEST -> VERIFIED -> CLOSED (or REJECTED/DUPLICATE).
- **Impact Triage:** Severity (Critical, High, Medium, Low) and Priority (Urgent, High, Medium, Low).
- **Traceability Linkage:** Direct 2-way links between defects, test cases, and test execution runs.
- **Collaboration:** Threaded comments and complete audit trail history of every status transition.

5. AI Intelligence Engine & CI/CD

INTELLIGENCE

Module 7: AI Intelligence Capabilities

- **AI Test Case Generator:** Converts unstructured user stories or PRDs into fully structured test cases with preconditions and steps.
- **AI Root-Cause Diagnostics:** Analyzes failed test logs and stack traces to identify code regressions and suggested fixes.
- **Duplicate Defect Detection:** Scans incoming defect reports against existing open bugs using semantic text similarity.
- **Executive QA Risk Reports:** Generates high-level release readiness summaries for engineering managers and stakeholders.

Module 8: CI/CD Automation & Integrations

- **Automated Test Ingestion:** Native reporters for Playwright, Cypress, Newman/Postman, and JUnit XML.
- **Scoped API Tokens:** Generate project-specific or org-wide API tokens for continuous integration pipelines.
- **Outgoing Webhooks:** Real-time webhook notifications for Slack, Jira, GitHub, or custom microservices on test completions.

Module 9: Executive Analytics & Quality Gates

Live charts showing overall pass percentage, test coverage by suite, defect density trends, flakiness index, and customizable quality gates that can block CI deployments if pass rates fall below 95%.

6. End-to-End User Flow (Step-by-Step)

USER JOURNEY

Phase 1: Project Setup & Team Onboarding

1. Sign in as Admin or QA Lead (e.g. sarah.lead@acme.staging.local).

2. Create or select an active Project (e.g. "Micky Mobile" with key "MI").

3. Define test environments (e.g., Staging, Production).

Phase 2: Authoring Test Repository

4. Navigate to Test Cases -> Create Suite ("User Profile").

5. Click "New Test Case" -> Enter Title, Preconditions, and Step-by-Step actions with Expected Results.

6. Save the test case (system assigns key MI-1, v1).

Phase 3: Launching Execution Run

7. Navigate to Test Runs -> Click "Create Test Run".

8. Provide run name ("Sprint 1 - Avatar Smoke Run") and select test case MI-1.

9. Click "Create Test Run" -> Open the execution workbench.

Phase 4: Executing Tests & Logging Defects

10. Click "Start Test Run" to transition state to IN_PROGRESS.

11. Click "Execute" on test case MI-1.

12. Verify each step: mark Pass/Fail, enter observed outcomes.

13. If a bug is discovered, click "Log Defect" to create ticket (e.g. MI-BUG-1) with linked evidence.

14. Complete the test run when all cases are verified.

Phase 5: Defect Retesting & Milestone Sign-Off

15. Developers move defect from NEW to IN_PROGRESS, apply fix, and mark READY_FOR_RETEST.

16. QA retests the test case, logs second attempt, and marks defect VERIFIED and CLOSED.

17. QA Lead reviews the Milestone progress bar and signs off on the release with 100% confidence.